

Knowledge Representation Methods

Representing Knowledge so that Computers can use it !

1 What is Knowledge Representation ?

- Knowledge Representation (KR) is a way to represent real-world knowledge in a form that a computer can understand, store and reason with.
- It provides a structured format to represent facts, rules, objects, relationships, etc.

"KR is the bridge between real world and the computer's reasoning."

2 Why is it Needed ?

- Real-world knowledge is complex and uncertain.
- Helps in **reasoning**, problem solving and decision making.
- Makes knowledge reusable and **maintainable**.
- Used in expert systems, robotics, NLP, etc.

3 Knowledge Representation Methods

Different methods are used depending on the type of knowledge and the application.

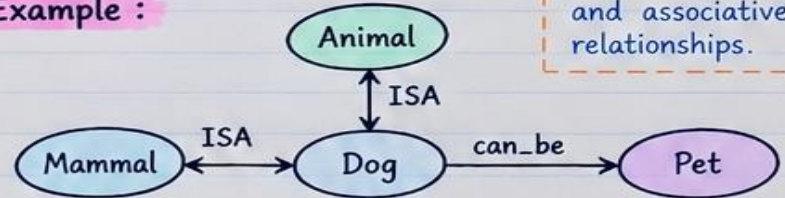
- 1 Semantic Networks
- 2 Frames
- 3 Logic (Propositional & First-order)
- 4 Production Rules
- 5 Scripts
- 6 Ontologies
- 7 Probabilistic Methods

No single method is best for all situations.
Choice depends on the problem domain.

4 1 Semantic Networks

- Represents knowledge as a graph with nodes and labeled links.
- Nodes represent objects or concepts.
- Links represent relationships.

Example :



Shows hierarchical and associative relationships.

5 2 Frames

- Represents knowledge using structured data structures called frames.
- A frame has slots (**attributes**) and **values**.
- Useful for representing stereotyped situations (objects, events).

Example :

Frame : Car

Slots :

- brand : Toyota
- color : Red
- no_of_wheels : 4
- type : Sedan

} Frame representation of a car object.

6 3 Logic (Propositional & First-Order Logic)

- Represents knowledge using logical statements (facts, predicates, quantifiers).
- Allows formal reasoning (inference).
- Propositional Logic** : deals with simple true/false statements.
- First-Order Logic (FOL)** : represents objects, properties and relationships using predicates.

Example (FOL) :

$\forall x (\text{Human}(x) \rightarrow \text{Mortal}(x))$
 $\text{Human}(\text{Socrates})$
 $\therefore \text{Mortal}(\text{Socrates})$

If x is a human then x is mortal.
Since Socrates is a human, therefore Socrates is mortal.

7 4 Production Rules

- Represents knowledge as IF-THEN rules.
- Widely used in expert systems.

Example :

IF Temperature > 100
THEN Boil = True

If temperature is greater than 100, then water boils.

8 5 Scripts

- Represents knowledge about common sequences of events (stereotyped situations).
- Useful for understanding everyday activities.

Example :

Script : Restaurant
1. Enter restaurant
2. Get seated
3. Order food
4. Eat food
5. Pay bill
6. Leave

Helps in understanding typical event sequences.

9 6 Ontologies

- A formal and explicit specification of shared concepts in a domain.
- Defines classes, relationships, constraints.
- Used in Semantic Web, knowledge sharing, AI.

Example :

Class : Student

- ↳ hasProperty : name, rollNo
- ↳ subclassOf : Person

10 7 Probabilistic Methods

- Represents uncertain or incomplete knowledge using probabilities.
- Commonly uses Bayesian Networks.

Example :

$P(\text{Rain}) = 0.3$
 $P(\text{WetGround} | \text{Rain}) = 0.8$

Helps in reasoning under uncertainty.

"Knowledge representation is the foundation of intelligent behavior."

Represent • Reason • Solve • Apply !

Forward Reasoning

"Start from what is known and infer new knowledge."

1 Definition

- Forward reasoning (also called **data-driven reasoning**) is an inference method that starts from **known facts** and repeatedly applies **inference rules** to **derive new facts** until the goal is reached or no more facts can be inferred.
- It is a **bottom-up approach** in reasoning.

2 Basic Idea

- Start with the initial set of **known facts**.
- Find rules whose **conditions** are satisfied by the current facts.
- Fire (apply)** those rules.
- Add the new facts to the knowledge base.
- Repeat the process until the goal is derived or no new facts can be inferred.

3 Rule Representation

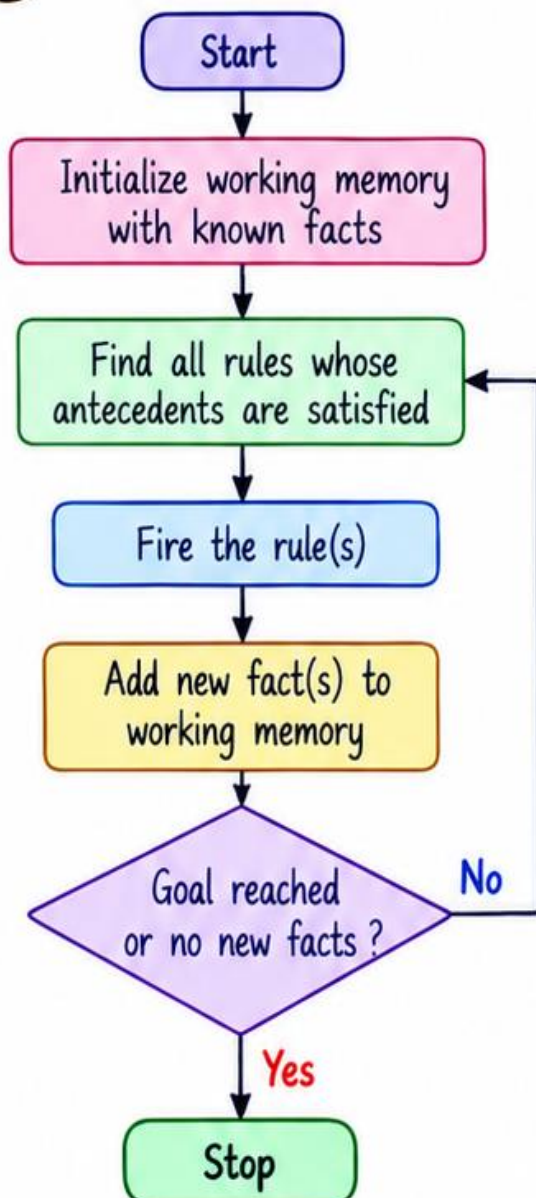
- Rules are written as **IF - THEN** statements.

IF condition(s)
THEN conclusion(s)

Example :

IF $A \wedge B$
THEN C

4 Algorithm



5 Example : Simple Inference

Knowledge Base

Facts :

1. A
2. B

Rules :

- R1 : IF A THEN C
R2 : IF B THEN C
R3 : IF C THEN D

Step-by-Step Execution

- 1 Initial facts = {A, B}
- 2 Apply R1 ($A \rightarrow C$) $\Rightarrow$ {A, B, C}
- 3 Apply R3 ($C \rightarrow D$) $\Rightarrow$ {A, B, C, D}
- 4 Goal D is reached ✓

Example : Medical Diagnosis (Brief)

Rules :

- R1 : IF Fever $\wedge$ Cough THEN Flu
R2 : IF Flu THEN Viral_Infection
R3 : IF Viral_Infection THEN Rest

Initial Facts : Fever, Cough

Goal : Rest

Inference Steps :

- 1 Facts = {Fever, Cough}
- 2 Apply R1 $\Rightarrow$ Flu
Facts = {Fever, Cough, Flu}
- 3 Apply R2 $\Rightarrow$ Viral_Infection
Facts = {Fever, Cough, Flu, Viral_Infection}
- 4 Apply R3 $\Rightarrow$ Rest
Goal Rest is reached ✓

6 Advantages

- ✓ Simple and intuitive.
- ✓ Good for problems with large amount of data.
- ✓ Can derive all possible conclusions.
- ✓ Widely used in expert systems.

7 Applications

- Expert systems (e.g., medical diagnosis)
- Decision support systems
- Configuration systems
- Natural language understanding
- Planning and scheduling
- Fault diagnosis

"From facts to knowledge – step by step."

Backward Reasoning

Artificial Intelligence
(7th Sem - GTU)

1) Introduction :

- Backward reasoning is a **goal-driven** reasoning technique.
- It starts with a query (goal) and works **backwards** to find the facts that can prove the goal.
- It uses rules to reduce the goal into sub-goals until all sub-goals are satisfied by known facts in the knowledge base.
- If all sub-goals are proved, the goal is true. If no rules can prove the goal, it fails.

2) Key Idea :

- Start with the goal (query).
- Find rules whose conclusion matches the goal.
- Replace the goal with the rule's antecedents (sub-goals).
- Repeat recursively until you reach known facts.

Backward reasoning is used in logic programming (e.g. Prolog) where the system tries to prove a query using a set of facts and rules.

3) Backward Reasoning vs Forward Reasoning.

Aspect	Backward Reasoning	Forward Reasoning
Strategy	Goal-driven	Data-driven
Start from	Goal (query)	Known facts
Direction	Works backwards	Works forwards
Used in	Prolog (logic programming)	Expert systems
More efficient when	Specific goal is given	Need to derive all possible conclusions

4) Process (General Steps)



5) Example (AI Domain) :

Knowledge Base (Rules & Facts) :

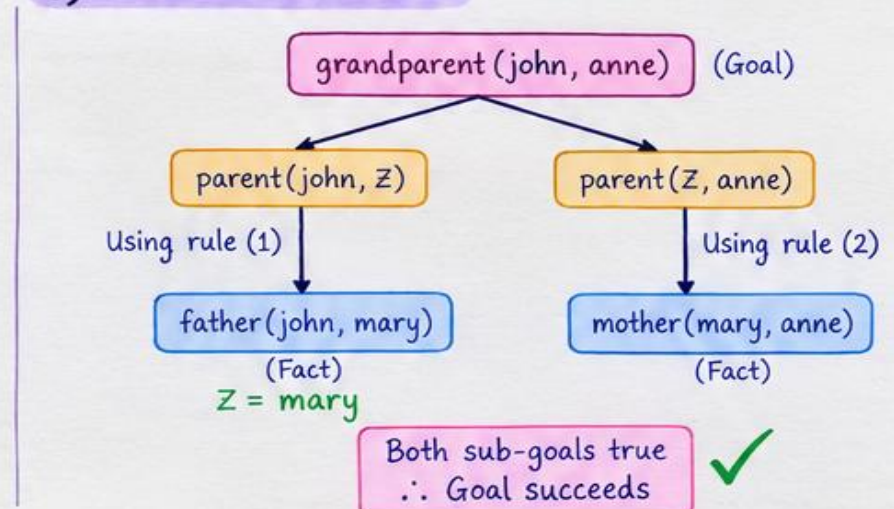
1. $\text{parent}(X,Y) :- \text{father}(X,Y).$
2. $\text{parent}(X,Y) :- \text{mother}(X,Y).$
3. $\text{grandparent}(X,Y) :- \text{parent}(X,Z), \text{parent}(Z,Y).$
4. $\text{father}(\text{john}, \text{mary}).$
5. $\text{mother}(\text{mary}, \text{anne}).$

Query : $?- \text{grandparent}(\text{john}, \text{anne}).$

Explanation :

- ① To prove $\text{grandparent}(\text{john}, \text{anne})$, we use rule (3).
- ② It requires two sub-goals : $\text{parent}(\text{john}, Z)$ and $\text{parent}(Z, \text{anne})$.
- ③ $\text{parent}(\text{john}, Z)$ is proved using rule (1) with fact $\text{father}(\text{john}, \text{mary}) \Rightarrow Z = \text{mary}$.
- ④ $\text{parent}(\text{mary}, \text{anne})$ is proved using rule (2) with fact $\text{mother}(\text{mary}, \text{anne})$.
- ⑤ Since both sub-goals are true, the query succeeds.
 $\therefore \text{john is a grandparent of anne.}$

6) Derivation Tree :



7) Prolog Program :

```

parent(X,Y) :- father(X,Y).
parent(X,Y) :- mother(X,Y).
grandparent(X,Y) :- parent(X,Z), parent(Z,Y).

father(john, mary).
mother(mary, anne).

?- grandparent(john, anne).
  
```

Output :
true. ✓

8) Applications :

- Expert systems
- Diagnostic systems (e.g. medical diagnosis)
- Query answering in knowledge bases
- Theorem proving

9) Advantages :

- ✓ Efficient when a specific goal is given.
- ✓ Avoids irrelevant facts/rules.
- ✓ Widely used in Prolog and logic-based systems.

10) Limitations :

- ✗ May be inefficient if many rules apply.
- ✗ Can lead to infinite loops (if not handled properly).
- ✗ Depends on the availability of suitable rules.

"Backward reasoning starts with a question and ends with an answer." 😊

Forward Reasoning - Example

AI
(7th Sem - GTU)

1) Idea :

- Start from known facts (in knowledge base).
- Apply rules to infer new facts.
- Continue forward until the goal is reached or no more facts can be inferred.
- It is a **data-driven** reasoning approach.

2) Example Problem :

Determine whether a person is a grandparent using forward reasoning.

3) Knowledge Base :

Facts :

- father(john, mary).
- mother(mary, anne).

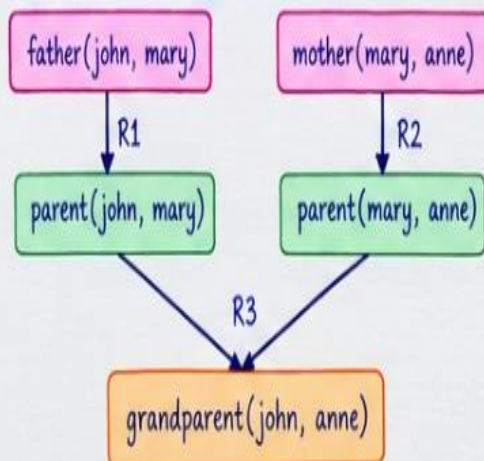
Rules :

- R1: parent(X, Y) :- father(X, Y).
 R2: parent(X, Y) :- mother(X, Y).
 R3: grandparent(X, Z) :-
 parent(X, Y), parent(Y, Z).

4) Forward Reasoning Steps :

Step	Inference	New Fact Derived	Known Facts After Step
1	From fact 1 using R1	parent(john, mary).	father(john, mary) parent(john, mary)
2	From fact 2 using R2	parent(mary, anne).	mother(mary, anne) parent(john, mary) parent(mary, anne)
3	Using R3 with parent(john, mary) and parent(mary, anne)	grandparent(john, anne).	All above facts + grandparent(john, anne)

5) Inference Diagram :



6) Result :

Using forward reasoning, we derived **grandparent(john, anne)** from the given facts.

7) Conclusion :

- Forward reasoning starts with facts and moves forward using rules.
- It is useful when we want to derive all possible conclusions (e.g. in expert systems, monitoring, diagnosis).

"Forward reasoning is driven by data." 😊

Backward Reasoning - Example

AI
(7th Sem - GTU)

1) Idea :

- Start from the goal (query).
- Look for rules whose conclusion matches the goal.
- Create sub-goals from the rule antecedents.
- Prove each sub-goal recursively until all are satisfied by known facts.
- It is a **goal-driven** reasoning approach.

2) Example Problem :

Prove the query **grandparent(john, anne)** using backward reasoning.

3) Knowledge Base :

Facts :

- father(john, mary).
- mother(mary, anne).

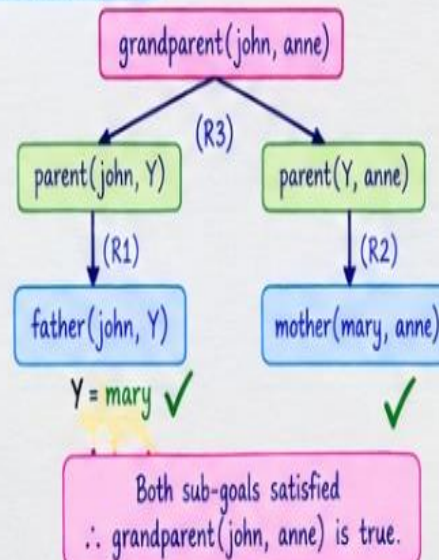
Rules :

- R1: parent(X, Y) :- father(X, Y).
 R2: parent(X, Y) :- mother(X, Y).
 R3: grandparent(X, Z) :-
 parent(X, Y), parent(Y, Z).

4) Backward Reasoning Steps :

Step	Goal (Query)	Action / Rule Used	Sub-goals	Result
1	grandparent(john, anne)	Use R3	parent(john, Y), parent(Y, anne)	New sub-goals generated
2	parent(john, Y)	Use R1	father(john, Y)	Fact matches: Y = mary
3	parent(mary, anne)	Use R2	mother(mary, anne)	Fact matches (known fact)
4	Both sub-goals true	—	—	grandparent(john, anne) proved ✓

5) Goal Tree :



6) Result :

The query **grandparent(john, anne)** is successfully proved using backward reasoning.

7) Conclusion :

- Backward reasoning starts with a goal and works backwards.
- It is useful when we want to check whether a specific query is true (e.g. in Prolog, theorem proving, query answering systems).

"Backward reasoning is driven by the goal." 😊

"Real world is complex, incomplete and uncertain - that's why knowledge representation is challenging."

Issues in Knowledge Representation

Challenges in representing real world knowledge in AI

Artificial Intelligence
(7th Sem - GTU)
(Prolog)

1) Introduction :

- Knowledge representation (KR) is the process of representing real-world knowledge in a form that a computer can use to reason and solve problems.
- However, there are several issues and challenges in knowledge representation due to the complexity, dynamic and uncertain nature of the real world.

2) Why are there Issues ?

- The real world is not always perfect, complete or static.
- Knowledge is often uncertain, ambiguous and context dependent.
- It is difficult to represent common sense, defaults, time, change and human reasoning.

3) Major Issues in Knowledge Representation

- 3.1 Representation of Incomplete Knowledge
- 3.2 Representation of Uncertain and Vague Knowledge
- 3.3 Handling of Inconsistency
- 3.4 Non-monotonic Reasoning (Handling Change)
- 3.5 Frame, Qualification and Context Problems
- 3.6 Representing Time and Actions
- 3.7 Scalability and Efficiency
- 3.8 Expressiveness vs. Computational Tractability
- 3.9 Common Sense Knowledge Representation
- 3.10 Domain Dependence

4) Details of Issues with Examples :

4.1 Incomplete Knowledge :

- In real world, we rarely have complete information.
- KR systems must handle missing facts.

Example :

We know that "Ramesh is a student" but we may not know his age or address.

Prolog Example :

```
student(ramesh).  
% age is unknown  
?- age(ramesh).  
false. % knowledge is incomplete
```

System should be able to reason even with partial information.

4.2) Uncertain and Vague Knowledge :

- Many real-world facts are uncertain or not precisely defined.
- Traditional logic (like Prolog) is binary (true/false), which is insufficient.

Example :

The statement "It is likely to rain tomorrow" is uncertain.

Handled using probabilistic reasoning or fuzzy logic.

4.3) Handling of Inconsistency :

- Knowledge from different sources may be contradictory.
- System should be able to detect and resolve inconsistencies.

Example :

Source 1: "Birds can fly."
Source 2: "Penguins are birds."
Source 3: "Penguins cannot fly."

Need mechanisms like default reasoning, exception handling or non-monotonic reasoning.

4.4) Non-monotonic Reasoning (Handling Change) :

- New information may invalidate previous conclusions.
- KR systems should allow withdrawal of conclusions.

Example :

Initially: $\neg \text{abnormal}(\text{bird}) \rightarrow \text{can_fly}(\text{bird})$
Later: penguin is known to be abnormal $\rightarrow$ cannot fly.

Prolog uses negation as failure which is non-monotonic.

4.5) Frame, Qualification and Context Problems :

- It is difficult to represent default properties and exceptions.
- Need to specify when a property holds (context).

Example :

Generally birds can fly, but not in all situations (e.g., injured bird, penguin, or a bird in a closed cage).

Frames and default reasoning are used to handle this issue.

4.6) Representing Time and Actions :

- Knowledge changes over time.
- Need to represent temporal information and actions.

Example :

At time t_1 : door is open.
At time t_2 : door is closed.

Requires temporal logic, situation calculus or event representation.

4.7) Scalability and Efficiency :

- As the amount of knowledge grows, reasoning becomes computationally expensive.
- Efficient indexing, inference and storage techniques are required.

Example :

In a large knowledge base (e.g., medical diagnosis), searching relevant facts can be time consuming.

4.8) Expressiveness vs. Computational Tractability :

- More expressive representations can model complex knowledge, but reasoning may become intractable.
- Need a balance between expressive power and efficiency.

Example :

First-Order Logic is more expressive than propositional logic, but reasoning is more complex.

4.9) Common Sense Knowledge Representation :

- Humans use common sense knowledge, which is hard to formalize.
- It includes implicit, everyday knowledge.

Example :

"If a person is hungry, he will eat." (common sense, not always explicit)

4.10) Domain Dependence :

- The choice of representation depends on the application domain.
- A single KR scheme may not be suitable for all domains.

Example :

Medical diagnosis, robotics, and natural language processing require different knowledge representation techniques.

Key Takeaway :

The main issue in knowledge representation is to effectively and efficiently model the real world with all its complexity, uncertainty and change.



"A good knowledge representation should model the real world, not an ideal world."



Predicate logic helps us represent knowledge using objects, properties and relationships.

Quantifiers in Predicate Logic

Talking about All and Some

Fundamentals of AI
(5th Sem Diploma - GTU)



Logic today
Intelligent systems
tomorrow!

1) What are Quantifiers ?

- Quantifiers are symbols used in predicate logic to specify the quantity of objects for which a predicate is true.
- They allow us to make general statements about a domain of discourse.
- Two main quantifiers are used:
 - Universal Quantifier ($\forall$)** - for all
 - Existential Quantifier ($\exists$)** - there exists

Quantifiers help us express "all" and "some" in logic.

2) Types of Quantifiers

Quantifier	Symbol	Meaning	Read as
Universal Quantifier	$\forall$	For all elements in the domain	"For all ..." "Every ..."
Existential Quantifier	$\exists$	There exists (at least one element)	"There exists ..." "Some ..." "At least one ..."

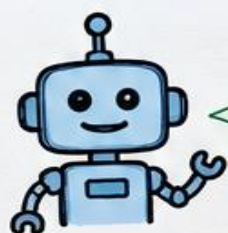
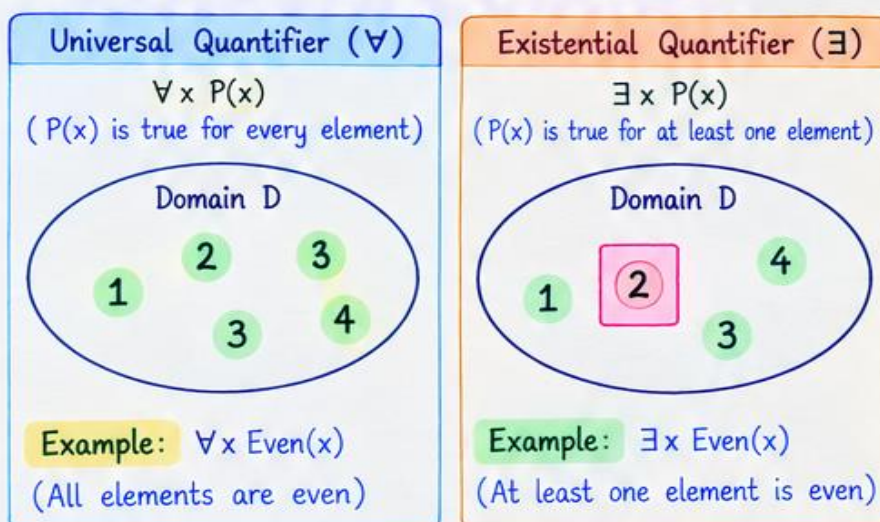
3) Domain and Predicate

- Domain:** set of all objects under consideration.
- Predicate:** a property or relation about the objects.

Example:

Let the domain be $D = \{1, 2, 3, 4\}$
Predicate $P(x)$: "x is even".

4) Representation using Diagrams



"Quantifiers make our knowledge more general and powerful!"

5) Examples

Statement	Predicate Logic	Meaning
All humans are mortal.	$\forall x (\text{Human}(x) \rightarrow \text{Mortal}(x))$	For every x, if x is a human then x is mortal.
There is a dog in the class.	$\exists x (\text{Dog}(x) \wedge \text{InClass}(x))$	At least one x is a dog and is in the class.
No student is absent.	$\forall x (\text{Student}(x) \rightarrow \neg \text{Absent}(x))$	For all x, if x is a student then x is not absent.
Some birds can fly.	$\exists x (\text{Bird}(x) \wedge \text{CanFly}(x))$	There exists at least one bird which can fly.

6) More Examples with Solution

A. Domain: $D = \{1, 2, 3, 4\}$ Predicate: $P(x)$: "x > 2"

- $\forall x P(x) \rightarrow$ Is it true?
Check: $1 > 2?$ (F) $\Rightarrow$ **False** So, $\forall x (x > 2)$ is false.
- $\exists x P(x) \rightarrow$ Is it true?
Check: $3 > 2?$ (T) $\Rightarrow$ **True** So, $\exists x (x > 2)$ is true.

B. Domain: $D = \{\text{Ramesh, Suresh, Meena}\}$
Predicate: $\text{Student}(x)$: "x is a student"

- $\forall x \text{ Student}(x) \rightarrow$ True (if all three are students)
- $\exists x \text{ Student}(x) \rightarrow$ True (if at least one is a student)

7) Quantifiers in Everyday Life

- $\forall x (\text{Plant}(x) \rightarrow \text{NeedsWater}(x))$
All plants need water.
- $\exists x (\text{Phone}(x) \wedge \text{Expensive}(x))$
There exists a phone which is expensive.
- $\forall x (\text{Student}(x) \rightarrow \text{Studies}(x))$
Every student studies.



"Logic is not just for machines, it helps us reason about the real world!"

8) Key Points to Remember

- $\forall$ means "for all" elements in the domain.
- $\exists$ means "there exists" at least one element.
- Quantifiers are used with predicates (not with individual constants).
- Order of quantifiers matters.
- They help us express general knowledge in AI.
- Used in knowledge representation, reasoning and expert systems.

9) Applications in AI

- Representing general knowledge (e.g., rules).
- Used in inference and theorem proving.
- Helps in knowledge bases and logical reasoning.
- Useful in natural language understanding.
- Applied in expert systems and intelligent agents.

From facts to reasoning
Quantifiers bridge the gap!



"The power of AI lies in representing the world correctly, and quantifiers help us do that."



“FOL helps us talk about objects, their properties and relationships.”

First Order Logic (FOL)

A language to represent and reason about the real world

Fundamentals of AI
(5th Sem Diploma – GTU)

Knowledge Representation
→ Better Reasoning
→ Smarter Systems

1) Introduction

- First Order Logic (FOL) is a formal language used to represent knowledge about **real world objects**, their **properties and relationships**.
- It is more expressive than propositional logic because it can talk about **individual objects** using **predicates and quantifiers**.
- FOL is widely used in Artificial Intelligence for knowledge representation, reasoning and in logic programming (e.g., Prolog).

Example :

In FOL we can say “Ramesh is a student.” instead of using just a single proposition.

2) Building Blocks of FOL

Component	Symbol / Form	Meaning	Example
Constants	a, b, Ramesh	Specific object in domain	Ramesh, 5
Variables	x, y, z	Represent objects (generic)	x, student
Predicates	P(x), Likes(x,y)	Properties or relations	Student(x)
Functions	f(x)	Map object to another object	father(x)
Quantifiers	$\forall$ (for all) $\exists$ (there exists)	Specify the quantity of objects	$\forall x P(x)$, $\exists x P(x)$
Logical Connectives	$\neg$, $\wedge$, $\vee$, $\rightarrow$, $\leftrightarrow$	Combine formulas	$P(x) \wedge Q(x)$

3) Predicates, Terms and Atoms

- Term**: constant, variable or function (Ramesh, x, father(x))
- Predicate**: represents a property or relation (Student(x))
- Atom**: a predicate with terms (Student(Ramesh))

Example :

Student(Ramesh) → atom
father(x) → term using function
Likes(Ramesh, Cricket) → atom with two terms

4) Quantifiers in FOL

- Universal Quantifier ($\forall$)**: true for all objects in the domain.
- Existential Quantifier ($\exists$)**: true for at least one object in the domain.

Example :

$\forall x \text{ Student}(x)$ → Every x is a student.
 $\exists x \text{ Likes}(x, \text{AI})$ → There exists at least one x who likes AI.

5) Syntax of FOL

- If P is a predicate and $t_1, t_2, \dots, t_n$ are terms, then $P(t_1, t_2, \dots, t_n)$ is a well-formed formula (WFF).
- If ϕ and ψ are formulas, then $\neg\phi$, $(\phi \wedge \psi)$, $(\phi \vee \psi)$, $(\phi \rightarrow \psi)$ are also formulas.
- If ϕ is a formula and x is a variable, then $\forall x \phi$ and $\exists x \phi$ are formulas.

Example :

$P(x)$ (atomic formula)
 $\neg P(x)$ (negation)
 $P(x) \wedge Q(x)$ (conjunction)
 $\forall x P(x)$ (universal quantification)

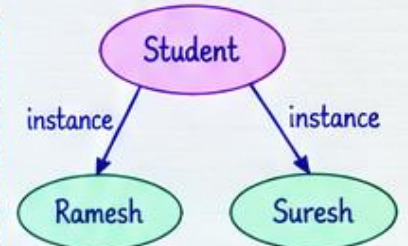
Well-formed formulas are built using these rules.

6) Example Representing a Small World

Let us represent some facts about students and courses.

Facts :

- Ramesh is a student.
- Suresh is a student.
- Ramesh likes AI.
- Suresh likes Math.
- Every student is interested in learning.



7) FOL Representation

English Statement	FOL Expression
Ramesh is a student.	Student(Ramesh)
Suresh is a student.	Student(Suresh)
Ramesh likes AI.	Likes(Ramesh, AI)
Suresh likes Math.	Likes(Suresh, Math)
Every student is interested in learning.	$\forall x (\text{Student}(x) \rightarrow \text{Interested}(x))$

8) Inference Example

From the facts, we can infer new knowledge.

Given :

- Student(Ramesh)
- $\forall x (\text{Student}(x) \rightarrow \text{Interested}(x))$

Inference :

∴ Interested(Ramesh)

If Ramesh is a student and every student is interested in learning, then Ramesh is interested in learning.

9) Representation in Prolog

- Prolog is based on First Order Logic.
- Facts are written as predicates.
- Rules are written using :- (which is like $\rightarrow$).
- Variables in Prolog start with uppercase letters.

Example in Prolog :

```
% Facts
student(ramesh).
student(suresh).
likes(ramesh, ai).
likes(suresh, math).
```

```
% Rule
interested(X) :- student(X).
```

Sample Query and Output :

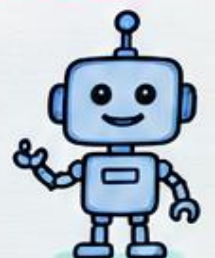
```
?- interested(ramesh).
true.
?- interested(suresh).
true.
```

Prolog uses First Order Logic for knowledge representation and logical inference.

10) Applications of FOL

- Knowledge representation in AI
- Expert systems
- Natural language understanding
- Automated reasoning and theorem proving
- Logic programming (Prolog)
- Planning and decision making
- Robotics and intelligent agents

FOL provides a simple yet powerful way to model real world knowledge.



“Logic is the language of intelligent behaviour.”



Represent → Reason → Solve → Intelligent Systems